

Module 1

Fundamentals of Software Testing and Quality Assurance

Role of testing in software development, Importance of processes in software quality assurance, Faults, errors, and failures in software testing, Limitations and challenges in software testing, Concepts of verification and validation in software quality assurance.

Software

- Software is a set of programs, instructions, and data that tell a computer what to do and how to do it.
- Software development life cycle:
 1. Planning
 2. Requirements analysis
 3. System design
 4. Implementation/coding
 5. Testing
 6. Deployment
 7. maintenance

Software Quality

- Software Quality means how well a software meets the user's needs, works correctly, and performs without problems.

1. Quality of Functionality (Does it work correctly?)

- No crashes
- Gives correct output
- Features work as expected
- Reliable results

2. Quality of Use (How well does it perform?)

- Speed
- Security
- Usability
- Performance
- Scalability

Five views of quality

1. Transcendental View:

- It defines quality as something that can be recognized through experience but is difficult to define.
- It can be applied to other complex areas of life.
- Quality is considered very ideal.

2. User View:

- It perceives quality as fitness for purpose.
- While evaluating the quality of a product one must ask the the key question: "Does the product satisfy user needs and expectations?"
- It concerns to the extent to which a product meets user needs and expectations.
- This view is highly personalized in nature

Five views of quality

3. Manufacturing View:

- Here quality is understood as conformance to the specifications.
- The quality level of a product is determined by the extent to which the product meets its specifications.
- A products quality can be incrementally enhanced by continuously improving the process.

4. Product View:

- Quality is viewed as tied to the inherent characteristics of the product.
- Product's inherent characteristics is internal qualities, determine its external qualities.

Five views of quality

5. Value Based View:

- Quality depends on the amount a customer is willing to pay for it.
- The value based view represents a merger of two independent concepts: excellence and worth.
- Quality is a measure of excellence and value is a measure of worth.

Reasons for Measuring quality:

Measurement allows us to establish baselines for qualities.

- Developers must know the minimum level of quality they must deliver for a product to be acceptable.
- Organizations make continuous improvements in their process models- and an improvement has a cost associated with it. Organizations need to know how much improvements in quality is achieved at a certain cost incurred due to process improvement.
- The present level of quality of a product needs to be evaluated so the need for improvements can be investigated.

Measuring Quality

- Measurement of User's View

- Its includes a number of quality factors such as functionality, reliability, and usability.
- A product may require multiple test cases for the same functionality if the functionality is to be performed in different execution environments.
- The ratio of the number of passed test cases to the total number of test cases designed to verify the functionalities is a measure of the functionalities delivered by the product.
- Usability is broken down into 3 sub characteristics:
 - Learnability
 - Understandability
 - operability

1. Learnability:
 - It is the average elapsed time for a typical user to gain a certain level of competence in using the product.
2. Understandability:
 - The average time needed by a typical user to gain a certain level of understanding of the product.
3. Operability:
 - The effort a user needs to actively use and control the software once they have learned how it works.

- Measurement of Manufacturer's View

- Defect Count: How many defects have been detected?
 - If a product is in operation, this count includes the defects detected during development and operation.
 - A defect count reflects the quality of work produced.
- Rework Cost: How much does it cost to fix the known defects?
 - The rework cost can be split into two parts as follows:
 1. Development Rework Cost: This is the rework cost incurred before a product is released to the customers.
 2. Operation Rework Cost: This is the rework cost incurred when a product is in operation.

MCCALL'S QUALITY FACTORS AND CRITERIA

Quality Factors

- A quality factor represents a behavioral characteristic of a system.
- McCall's quality factors emphasize more on the quality levels of a product delivered by an organization and the quality levels of a delivered product relevant to product maintenance.
- high-level quality factors are:
 - Correctness
 - Reliability
 - Efficiency
 - Testability
 - Portability
 - Reusability.

Correctness:

- A software system is expected to meet the explicitly specified functional requirements and the implicitly expected nonfunctional requirements.
- If a software system satisfies all the functional requirements, the system is said to be correct.
- A correct software system may still be unacceptable to customers if the system fails to meet unstated requirements, such as stability, performance, and scalability.

Reliability:

- It is difficult to construct large software systems which are correct. A few functions may not work in all execution scenarios, and, therefore, the software is considered to be incorrect. However, the software may still be acceptable to customers because the execution scenarios causing the system to fail may not frequently occur when the system is deployed. Moreover, customers may accept software failures once in a while.
- Customers may still consider an incorrect system to be reliable if the failure rate is very small and it does not adversely affect their mission objectives.
- Reliability is a customer perception, and an incorrect software can still be considered to be reliable.

Efficiency:

- Efficiency concerns to what extent a software system utilizes resources, such as computing power, memory, disk space, communication bandwidth, and energy.
- A software system must utilize as little resources as possible to perform its functionalities.
- For example, by utilizing less communication bandwidth a base station in a cellular telephone network can support more users.

Integrity:

- A system's integrity refers to its ability to withstand attacks to its security.
- In other words, integrity refers to the extent to which access to software or data by unauthorized persons or programs can be controlled.
- Integrity has assumed a prominent role in today's network-based applications.
- Integrity is also an issue in multiuser systems.

Flexibility:

- Flexibility is reflected in the cost of modifying an operational system.
- As more and more changes are effected in a system throughout its operational phase, subsequent changes may cost more and more.
- If the initial design is not flexible, it is highly likely that subsequent changes are very expensive.
- In order to measure the flexibility of a system, one has to find an answer to the question: How easily can one add a new feature to a system?

Maintainability:

- It refers to the upkeep of products in response to deterioration of their components due to continued use of the products.
- Maintainability refers to how easily and inexpensively the maintenance tasks can be performed.
- For software products, there are three categories of maintenance activities:
 1. Corrective :
 - It refers to the removal of defects existing in an in-service software.
 - The existing defects might have been known at the time of release of the product or might have been introduced during maintenance
 2. Adaptive
 - concerns adjusting software systems to changes in the execution environment.
 3. Perfective
 - It concerns modifying a software system to improve some of its qualities.

Testability:

- It is important to be able to verify every requirement, both explicitly stated and simply expected.
- Testability means the ability to verify requirements.
- At every stage of software development, it is necessary to consider the testability aspect of a product.
- Specifically, for each requirement we try to answer the question: What procedure should one use to test the requirement, and how easily can one verify it?
- To make a product testable, designers may have to instrument a design with functionalities not available to the customer.

Portability:

- Portability of a software system refers to how easily it can be adapted to run in a different execution environment.
- An execution environment is a broad term encompassing hardware platform, operating system, distributedness, and heterogeneity of the hardware system.
- Portability is important for developers because a minor adaptation of a system can increase its market potential.
- Moreover, portability gives customers an option to easily move from one execution environment to another to best utilize emerging technologies in furthering their business.
- Good design principles such as modularity facilitate portability.
- For example, all environment-related computations can be localized in a few modules so that those can be easily identified and modified to port the system to another environment.

Reusability:

- If a significant portion of one product can be reused, maybe with minor modification, in another product.
- It may not be economically viable to reuse small components.
- Reusability saves the cost and time to develop and test the component being reused.
- In the field of scientific computing, mathematical libraries are commonly reused.
- Reusability is not just limited to product parts, rather it can be applied to processes as well.
- For example, we are very much interested in developing good processes that are largely repeatable.

Interoperability:

- In this age of computer networking, isolated software systems are turning into a rarity.
- Today's software systems are coupled at the input–output level with other software systems.
- Intuitively, interoperability means whether or not the output of one system is acceptable as input to another system; it is likely that the two systems run on different computers interconnected by a network.
- When we consider Internet-based applications and wireless applications, the need for interoperability is simply overriding.
- For example, users of document processing packages, such as LaTeX and Microsoft Word, want to import a variety of images produced by different graphics packages. Therefore, the graphics packages and the document processing packages must be interoperable.
- Another example of interoperability is the ability to roam from one cellular phone network in one country to another cellular network in another country.

quality factors defined have been grouped into three broad categories as follows:

- Product operation
- Product revision
- Product transition

TABLE 17.2 Categorization of McCall's Quality Factors

Quality Categories	Quality Factors	Broad Objectives
Product operation	Correctness	Does it do what the customer wants?
	Reliability	Does it do it accurately all of the time?
	Efficiency	Does it quickly solve the intended problem?
	Integrity	Is it secure?
	Usability	Can I run it?
Product revision	Maintainability	Can it be fixed?
	Testability	Can it be tested?
	Flexibility	Can it be changed?
Product transition	Portability	Can it be used on another machine?
	Reusability	Can parts of it be reused?
	Interoperability	Can it interface with another system?

Quality Criteria

- A quality criterion is an attribute of a quality factor that is related to software development.
- It is a high-level goal or characteristic that a software product must achieve to satisfy the end user.
- For example, modularity is an attribute of the architecture of a software system. A highly modular software allows designers to put cohesive components in one module, thereby increasing the maintainability of the system.

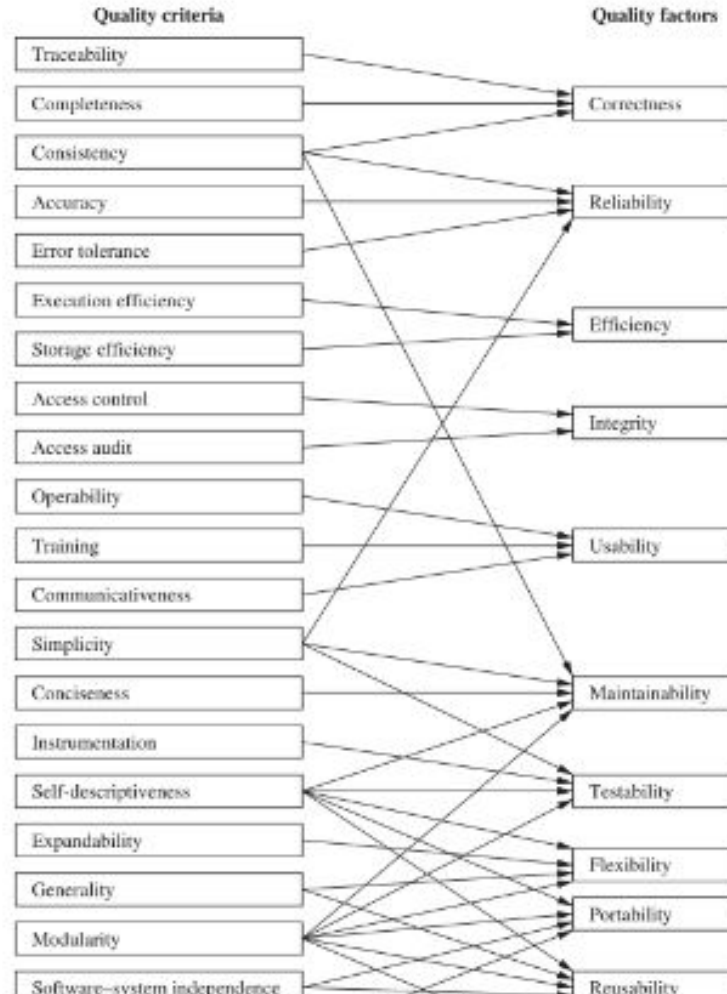
TABLE 17.3 McCall's Quality Criteria

Quality Criteria	Definition
Access audit	Ease with which software and data can be checked for compliance with standards or other requirements
Access control	Provisions for control and protection of the software and data
Accuracy	Precision of computations and output
Communication commonality	Degree to which standard protocols and interfaces are used
Completeness	Degree to which a full implementation of the required functionalities has been achieved
Communicativeness	Ease with which inputs and outputs can be assimilated
Conciseness	Compactness of the source code, in terms of lines of code

Consistency	Use of uniform design and implementation techniques and notation throughout a project
Data commonality	Use of standard data representations
Error tolerance	Degree to which continuity of operation is ensured under adverse conditions
Execution efficiency	Run time efficiency of the software
Expandability	Degree to which storage requirements or software functions can be expanded
Generality	Breadth of the potential application of software components
Hardware independence	Degree to which the software is dependent on the underlying hardware
Instrumentation	Degree to which the software provides for measurement of its use or identification of errors

Modularity	Provision of highly independent modules
Operability	Ease of operation of the software
Self-documentation	Provision of in-line documentation that explains implementation of components
Simplicity	Ease with which the software can be understood.
Software system independence	Degree to which the software is independent of its software environment—nonstandard language constructs, operating system, libraries, database management system, etc.
Software efficiency	Run time storage requirements of the software
Traceability	Ability to link software components to requirements
Training	Ease with which new users can use the system

Relationship b



riteria

Quality Metrics

- The high-level quality factors cannot be measured directly.
- For example, we cannot directly measure the testability of a software system. Neither can testability be expressed in “yes” or “no” terms.
- The degree of testability can be assessed by associating with testability a few quality metrics, namely, simplicity, instrumentation, self-descriptiveness, and modularity.
- A quality metric is a measure that captures some aspect of a quality criterion.
- One or more quality metrics should be associated with each criterion.

- The metrics can be derived as follows:
 - Formulate a set of relevant questions concerning the quality criteria and seek a “yes” or “no” answer for each question.
 - Divide the number of “yes” answers by the number of questions to obtain a value in the range of 0 to 1. The resulting number represents the intended quality metric.
- For example, we can ask the following question concerning the self-descriptiveness of a product:

“ Is all documentation written clearly and simply such that procedures, functions, and algorithms can be easily understood? “

Another question concerning self-descriptiveness is:

“Is the design rationale behind a module clearly understood?”

- Different questions can have different degrees of importance in the computation of a metric, and, therefore, individual “yes” answers can be differently weighted in the above computation.

ISO 9126 defines 6 broad quality characteristics:

1. Functionality

- Refers to the capability of the software to provide functions that meet stated and implied needs.

2. Reliability

- Measures the ability of the software to maintain its performance under specified conditions over time.

3. Usability

- The ease with which users can learn, operate, prepare inputs, and interpret outputs of the software.

ISO 9126 defines 6 broad quality characteristics:

4. Efficiency

- The software's ability to provide appropriate performance relative to the amount of resources used.

5. Maintainability

- The ease with which the software can be modified to correct faults, improve performance, or adapt to a changed environment.

6. Portability

- The ease with which the software can be transferred from one environment to another.

Role of Testing

- Achieving and assessing the quality of a software product.
- Improvements to the quality of products occurs during the test-find defects -fix cycle.
- How good our system is when we perform system level tests before releasing the product.
- According Friedman and Voas: **Software testing is a verification process for software quality assessment and improvement.**

Software Quality Assessment:

- Using these the practitioners want to identify as many as faults as possible so that those faults are fixed at an early stage of software development.
- Both these must be performed repeatedly and alternatively.
- The activities for software quality assessment can be divided into 2 broad categories:
 1. Static Analysis
 2. Dynamic Analysis

1. Static Analysis:

- Static activities check the quality of the software without running the program.
- Its based on the examination of a number of documents , namely requirements documents, software models, design documents, and source code.
- Traditional statistical analysis include code review, insection, walk through , algorithm analysis, and proof of correctness.
- It does not involve actual execution of code instead it examines code and reasons over all behaviors that might arise during run time.
- Compiler optimizations are standard static analysis

Software Quality Assessment

2. Dynamic Analysis:

- It includes actual program execution in order to expose possible program failures.
- The behavioural and performance properties are also observed.
- Programs are executed with both typical and carefully chosen input values.

OBJECTIVES OF TESTING

- The stakeholders in a test process are the programmers, the test engineers, the project managers, and the customers.
- A stakeholder is a person or an organization who influences a system's behaviors or who is impacted by that system

Different stakeholders view a test process from different perspectives as explained below:

- It does work:
 - While implementing a program unit, the programmer may want to test whether or not the unit works in normal circumstances.
 - The programmer gets much confidence if the unit works to his or her satisfaction.
 - The same idea applies to an entire system as well—once a system has been integrated, the developers may want to test whether or not the system performs the basic functions.
 - Here, for the psychological reason, the objective of testing is to show that the system works, rather than it does not work.

- It does not work:
 - Once the programmer (or the development team) is satisfied that a unit (or the system) works to a certain degree, more tests are conducted with the objective of finding faults in the unit (or the system).
 - Here, the idea is to try to make the unit (or the system) fail.
- Reduce the risk of failure:
 - Most of the complex software systems contain faults, which cause the system to fail from time to time.
 - This concept of “failing from time to time” gives rise to the notion of failure rate.
 - As faults are discovered and fixed while performing more and more tests, the failure rate of a system generally decreases.
 - Thus, a higher level objective of performing tests is to bring down the risk of failing to an acceptable level.

- Reduce the cost of testing:
 - The different kinds of costs associated with a test process include
 - the cost of designing, maintaining, and executing test cases,
 - the cost of analyzing the result of executing each test case,
 - the cost of documenting the test cases, and
 - the cost of actually executing the system and documenting it.
- producing a small number of arbitrary test cases is not a good way of saving cost.
- The highest level of objective of performing tests is to produce low-risk software with fewer number of test cases.
- This idea leads us to the concept of effectiveness of test cases.
- Test engineers must therefore judiciously select fewer, effective test cases.

WHAT IS A TEST CASE?

- most basic form, a test case is a simple pair of < input, expected outcome >.
- Eg: If a program under test is expected to compute the square root of nonnegative numbers,

TB1: < 0, 0 > ,

TB2: < 25, 5 > ,

TB3: < 40, 6.3245553 > ,

TB4: < 100.5, 10.024968 > .

- In stateless systems, where the outcome depends solely on the current input, test cases are very simple in structure.
- In state-oriented systems, where the program outcome depends both on the current state of the system and the current input, a test case may consist of a sequence of < input, expected outcome > pairs.

TS₁: < check balance, \$500.00 >, < withdraw, "amount?" >, < \$200.00, "\$200.00" >, < check balance, \$300.00 > .

Figure 1.4 Example of a test case with a sequence of < input, expected outcome > .

Verification and Validation

- Verification
 - Evaluating a software system by determining whether the product of a given development phase satisfies the requirements established before the start of that phase.
 - Activities that check the correctness of a development phase are called verification phase.
- Validation
 - Helps in confirming that a product meets its intended use.
 - It aims at confirming that a product meets its customers expectations.
 - It focuses on the final product.

- **Verification** activities aim at confirming that one is **building the product correctly** whereas **validation** activities aim at confirming that one is **building the correct product**.
- **Verification** activities **review interim work products** such as requirements specification, design, code and user manual during project life cycle to ensure there quality. **Validation** is performed at the **end of the system** development to check whether it meets customer requirement.
- Verification activities are performed on interim products but validation is performed on the entire system by running the system in actual environment.

FAILURE, ERROR, FAULT, AND DEFECT

- Failure: A failure is said to occur whenever the external behavior of a system does not conform to that prescribed in the system specification.
- Error: An error is a state of the system. In the absence of any corrective action by the system, an error state could lead to a failure which would not be attributed to any event subsequent to the error.
- Fault: A fault is the adjudged cause of an error.